

UNIT – 4

LEETCODE EXERCISES

1. Delete Duplicate Emails

Table: Person

Column Name	Type
id	int
email	varchar

id is the primary key (column with unique values) for this table. Each row of this table contains an email. The emails will not contain uppercase letters.

Problem Summary

Write a solution to delete all duplicate emails, keeping only one unique email with the smallest id.

SQL Schema

```
Create table If Not Exists Person (Id int, Email varchar(255));
```

```
Truncate table Person;
```

```
insert into Person (id, email) values ('1', 'john@example.com');
```

```
insert into Person (id, email) values ('2', 'bob@example.com');
```

```
insert into Person (id, email) values ('3', 'john@example.com');
```

Person

id	email
1	john@example.com
2	bob@example.com
3	john@example.com

MySQL Solution

</> Code



MySQL Auto



```
1 # Write your MySQL query statement below
2 delete p1
3 from Person p1, Person p2
4 where p1.Email = p2.Email and p1.Id > p2.Id
```

☒ Testcase | Test Result

Output

id email
-- -----
1 john@example.com
2 bob@example.com

2. Customers Who Never Order

Table: Customers

Column Name	Type
Id	int
Name	varchar

Table: Orders

Column Name	Type
Id	int
CustomerId	int

Problem Summary

Suppose that a website contains two tables, the Customers table and the Orders table. Write a SQL query to find all customers who never order anything.

SQL Schema

```
Create table If Not Exists Customers (Id int, Name varchar(255));
```

```
Create table If Not Exists Orders (Id int, CustomerId int);
```

```
Truncate table Customers;
```

```
insert into Customers (Id, Name) values ('1', 'Joe'), ('2', 'Henry'), ('3', 'Sam'), ('4', 'Max');
```

```
Truncate table Orders;
```

```
insert into Orders (Id, CustomerId) values ('1', '3'), ('2', '1');
```

Customers

Id	Name
1	Joe
2	Henry
3	Sam
4	Max

Orders

Id	CustomerId
1	3
2	1

MySQL Solution

</> Code

MySQL Auto

```
1 # Write your MySQL query statement below
2 select c.Name as Customers
3 from Customers c
4 where c.Id not in (select CustomerId from Orders);
```

Testcase Test Result

Orders =

id	customerId
1	3
2	1

3. Biggest Single Number

Table: my_numbers

Column Name	Type
num	int

Problem Summary

Table my_numbers contains many numbers in column num including duplicated ones. Write a SQL query to find the biggest number, which only appears once. If there is no such number, output null.

SQL Schema

Create table If Not Exists my_numbers (num int);

Truncate table my_numbers;

insert into my_numbers (num) values ('8'), ('8'), ('3'), ('3'), ('1'), ('4'), ('5'), ('6');

my_numbers

num
8
8
3
3
1
4
5
6

MySQL Solution

</> Code

MySQL  Auto    

```
1 # Write your MySQL query statement below
2 SELECT MAX(num) AS num
3 FROM (
4     SELECT num
5     FROM MyNumbers
6     GROUP BY num
7     HAVING COUNT(num) = 1
8 ) n;
```

☒ Testcase   Test Result  

Output

num

6

4. Game Play Analysis I

Table: Activity

Column Name	Type
player_id	int
device_id	int
event_date	date
games_played	int

(player_id, event_date) is the primary key of this table.

Problem Summary

Write an SQL query that reports the first login date for each player.

SQL Schema

Create table If Not Exists Activity (player_id int, device_id int, event_date date, games_played int);

Truncate table Activity;

insert into Activity (player_id, device_id, event_date, games_played) values ('1', '2', '2016-03-01', '5');

insert into Activity (player_id, device_id, event_date, games_played) values ('1', '2', '2016-05-02', '6');

insert into Activity (player_id, device_id, event_date, games_played) values ('2', '3', '2017-06-25', '1');

insert into Activity (player_id, device_id, event_date, games_played) values ('3', '1', '2016-03-02', '0');

insert into Activity (player_id, device_id, event_date, games_played) values ('3', '4', '2018-07-03', '5');

Activity

player_id	device_id	event_date	games_played
1	2	2016-03-01	5
1	2	2016-05-02	6
2	3	2017-06-25	1
3	1	2016-03-02	0
3	4	2018-07-03	5

MySQL Solution

</> Code

MySQL Auto

```
1 # Write your MySQL query statement below
2 select a.player_id, min(a.event_date) as first_login
3 from Activity a
4 group by a.player_id;
```

☒ Testcase | >_ Test Result

Output

player_id	first_login
1	2016-03-01
2	2017-06-25
3	2016-03-02

5. Game Play Analysis II

Table: Activity

Column Name	Type
player_id	int
device_id	int
event_date	date
games_played	int

(**player_id**, **event_date**) is the primary key of this table.

Problem Summary

Write a SQL query that reports the device that is first logged in for each player.

SQL Schema

Create table If Not Exists Activity (player_id int, device_id int, event_date date, games_played int);

Truncate table Activity;

insert into Activity (player_id, device_id, event_date, games_played) values ('1', '2', '2016-03-01', '5');

insert into Activity (player_id, device_id, event_date, games_played) values ('1', '2', '2016-05-02', '6');

insert into Activity (player_id, device_id, event_date, games_played) values ('2', '3', '2017-06-25', '1');

insert into Activity (player_id, device_id, event_date, games_played) values ('3', '1', '2016-03-02', '0');

insert into Activity (player_id, device_id, event_date, games_played) values ('3', '4', '2018-07-03', '5');

Activity

player_id	device_id	event_date	games_played
1	2	2016-03-01	5
1	2	2016-05-02	6
2	3	2017-06-25	1
3	1	2016-03-02	0
3	4	2018-07-03	5

MySQL Solution

```
</> Code
MySQL v Auto
1 # Write your MySQL query statement below
2 select player_id, device_id
3 from Activity
4 where (player_id, event_date) in
5      (select player_id, min(event_date)
6       from Activity
7       group by player_id);
```

Testcase | Test Result

Output

player_id	device_id
1	2
2	3
3	1

6. Game Play Analysis III

Table: Activity

Column Name	Type
player_id	int
device_id	int
event_date	date
games_played	int

(player_id, event_date) is the primary key of this table.

Problem Summary

Write a solution to report for each player and date, how many games played so far by the player. That is, the total number of games played by the player until that date.

SQL Schema

Create table If Not Exists Activity (player_id int, device_id int, event_date date, games_played int);

Truncate table Activity;

insert into Activity (player_id, device_id, event_date, games_played) values ('1', '2', '2016-03-01', '5');

insert into Activity (player_id, device_id, event_date, games_played) values ('1', '2', '2016-05-02', '6');

insert into Activity (player_id, device_id, event_date, games_played) values ('1', '3', '2017-06-25', '1');

insert into Activity (player_id, device_id, event_date, games_played) values ('3', '1', '2016-03-02', '0');

insert into Activity (player_id, device_id, event_date, games_played) values ('3', '4', '2018-07-03', '5');

Activity

player_id	device_id	event_date	games_played
1	2	2016-03-01	5
1	2	2016-05-02	6
1	3	2017-06-25	1
3	1	2016-03-02	0
3	4	2018-07-03	5

MySQL Solution

</> Code

MySQL Auto

```
1 # Write your MySQL query statement below
2 SELECT
3     a.player_id,
4     a.event_date,
5     SUM(b.games_played) games_played_so_far
6 FROM
7     Activity a LEFT JOIN Activity b ON
8     b.player_id = a.player_id AND
9     b.event_date <= a.event_date
10 GROUP BY
11     a.player_id,
12     a.event_date;
```

Testcase Test Result

Output

player_id	event_date	games_played_so_far
1	2016-03-01	5
1	2016-05-02	11
2	2017-06-25	1
3	2016-03-02	0
3	2018-07-03	5

7. Exchange Seats

Table: Seat

Column Name	Type
id	int
student	varchar

id is the primary key for this table.

Problem Summary

Write a solution to swap the seat id of every two consecutive students. If the number of students is odd, the id of the last student is not swapped.

SQL Schema

```
Create table If Not Exists Seat (id int, student varchar(255));
```

```
Truncate table Seat;
```

```
insert into Seat (id, student) values ('1', 'Abbot');
```

```
insert into Seat (id, student) values ('2', 'Doris');
```

```
insert into Seat (id, student) values ('3', 'Emerson');
```

```
insert into Seat (id, student) values ('4', 'Green');
```

```
insert into Seat (id, student) values ('5', 'Jeames');
```

Seat

id	student
1	Abbot
2	Doris
3	Emerson
4	Green
5	Jeames

MySQL Solution

</> Code

MySQL  Auto

```
1 # Write your MySQL query statement below
2 SELECT
3     a.id id,
4     COALESCE(b.student, a.student) student
5 FROM
6     seat a LEFT JOIN seat b
7     ON b.id = (CASE
8         WHEN a.id%2=1 THEN a.id+1
9         ELSE a.id-1
10        END)
11 ORDER BY a.id
```

☒ Testcase |  Test Result

Output

id	student
1	Doris
2	Abbot
3	Green
4	Emerson
5	Jeames

8. Triangle Judgement

Table: triangle

Column Name	Type
x	int
y	int
z	int

Problem Summary

Write a query to judge whether three line segments (x, y, z) can form a triangle.

SQL Schema

Create table If Not Exists triangle (x int, y int, z int);

Truncate table triangle;

insert into triangle (x, y, z) values ('13', '15', '30');

insert into triangle (x, y, z) values ('10', '20', '15');

triangle

x	y	z
13	15	30
10	20	15

MySQL Solution

```
</>Code
MySQL  Auto
1  # Write your MySQL query statement below
2  select *,
3     if(x+y>z and x+z>y and y+z>x, 'Yes', 'No') as triangle
4  from triangle;
```

Output

x	y	z	triangle
13	15	30	No
10	20	15	Yes



9. Consecutive Numbers

Table: Logs

Column Name	Type
id	int
num	int

id is the primary key for this table.

Problem Summary

Find all numbers that appear at least three times consecutively.

SQL Schema

Create table If Not Exists Logs (id int, num int);

Truncate table Logs;

insert into Logs (id, num) values ('1', '1');

insert into Logs (id, num) values ('2', '1');

insert into Logs (id, num) values ('3', '1');

insert into Logs (id, num) values ('4', '2');

insert into Logs (id, num) values ('5', '1');

insert into Logs (id, num) values ('6', '2');

insert into Logs (id, num) values ('7', '2');

Logs

id	num
1	1
2	1
3	1
4	2
5	1
6	2
7	2

MySQL Solution

</> Code

MySQL   Auto

```
1 # Write your MySQL query statement below
2 SELECT DISTINCT L1.NUM ConsecutiveNums
3 FROM Logs l1, Logs l2, LOGS L3
4 WHERE l1.Id = l2.Id - 1
5       AND L2.ID=L3.ID - 1
6       AND l1.Num = l2.Num
7       AND L2.NUM = L3.NUM;
```

 Testcase |  Test Result

Output

ConsecutiveNums
1

10. Tree Node

Table: Tree

Column Name	Type
id	int
p_id	int

id is the primary key for this table.

Problem Summary

Write a solution to report the type of each node in the tree ("Root", "Inner", or "Leaf").

SQL Schema

```
Create table If Not Exists Tree (id int, p_id int);
```

```
Truncate table Tree;
```

```
insert into Tree (id, p_id) values ('1', NULL);
```

```
insert into Tree (id, p_id) values ('2', '1');
```

```
insert into Tree (id, p_id) values ('3', '1');
```

```
insert into Tree (id, p_id) values ('4', '2');
```

```
insert into Tree (id, p_id) values ('5', '2');
```

Tree

id	p_id
1	null
2	1
3	1
4	2
5	2

MySQL Solution

 Code

MySQL  Auto

```
1 # Write your MySQL query statement below
2 SELECT
3     id,
4     (CASE
5         WHEN p_id IS NULL THEN "Root"
6         WHEN id IN (SELECT DISTINCT p_id FROM tree) THEN "Inner"
7         ELSE "Leaf"
8     END) Type
9 FROM tree;
```

☒ Testcase |  Test Result

Output

id	Type
1	Root
2	Inner
3	Leaf
4	Leaf
5	Leaf